
B28: SQL SuperHero Program : Session 46

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>

12 May 2025 at 04:51

Bcc: satishhattekar1997@gmail.com

As we discussed in the session, avoiding overhead on CPU is the most important task on which we should take care, avoiding overhead on Memory is more about DBA involvement.

Remember few tips :

1. Avoid loops
2. Perform Direct-Path INSERT if possible
3. Gather Statistics after Loading each Table
4. Use WITH to Split Complex Queries
5. Reduce Data as Early as Possible
6. Take Care of OR in WHERE Condition
7. Avoid Functions in WHERE Condition
8. Drop Unnecessary Indexes

Of course, these performance tips are not a complete reference for performance tuning of any jobs. But they help you to reduce the load times of many sqls. Few tips to remember:

1. Create an index on very large tables (>1,000,000 rows)
2. Use SELECT fields instead of SELECT *
3. Run your query during off-peak hours
4. Use INNER JOIN instead of WHERE for joining tables
5. Minimize the use of subqueries
6. Use derived and temporary tables
7. Choose LEFT/RIGHT join when possible over INNER join for the same output
8. Use EXISTS() instead of COUNT() to find an element in the table
9. Avoid SELECT DISTINCT where possible
10. Use GROUP BY over window functions
11. Use UNION ALL instead of UNION where possible
12. Use UNION instead of OR if possible
13. Use LIMIT to sample query results
14. Drop the index before loading bulk data
15. Use materialized views instead of views
16. Partition large tables to improve query performance
18. Try to avoid != or <> (not equal) operators
19. Try to avoid Scalar functions in WHERE and JOIN clauses
20. Try to avoid Wildcards at the beginning of LIKE operator
21. Use appropriate data types and collation for columns
22. Encapsulate complex queries in stored procedures

=====

Explain Plan

An explain plan is a prediction about what might happen when you run a query. To improve the performance of any code, you need to know what the query really did. You need the exact sequence of steps the database took to run the query, how many rows each operation returned, the I/O needed to process it, and how long it took to execute. The EXPLAIN PLAN statement displays execution plans chosen by the Oracle optimizer for SELECT, UPDATE, INSERT, and DELETE statements.

The row source tree is the core of the execution plan. It shows the following information:

- An ordering of the tables referenced by the statement
- An access method for each table mentioned in the statement
- A join method for tables affected by join operations in the statement
- Data operations like filter, sort, or aggregation

When you write some SQL, Oracle runs it through the optimizer to determine the best access path. This involves taking the parsed representation of the SQL Statement, taking statistics about the data (which have been gathered in advance), generating multiple plans, comparing the plans and finally selecting the plan with the lowest “cost” and using it to access the data.

Oracle does not always get it right. In fact, it frequently gets the plans wrong by trying to find reasonable execution plans in a reasonable amount of time. If the optimizer gets the plan wrong, then the next time the SQL is parsed, it will get it wrong again. and again.

Oracle weighs up a lot of information about your query to determine the optimal method of access. That takes time, and for complex SQL with many joins it may take longer to work out the best plan than it would take to just go and get the data with a sub-optimal plan. Oracle takes a pragmatic approach (Define the problem, search for existing solution rather than understand the basics) and, once it reaches its (by default) 2000th-ish “best guess” it just goes and gets the data.

Do you think that 2,000 guesses is a lot? but a number of possibilities that the optimizer must consider to determine the best plan.

For example, a 5 table join, the maths goes something like this:

Get a Join Order – Should we join table A-to-B-to-C-to-D-to-E, or A-to-B-to-C-to-E-to-D, or... That works out at 5! (factorial), or 120 possibilities.

Get a Join Method – There’s 3 options; Nested Loop, Hash Join and Merge Join. So that’s A-to-B x 3 + B-to-C x 3 + C-to-D x 3 and D-to-E x 3. $3 \times 3 \times 3 \times 3$ is 81 possibilities.

Get a Data Access path for each table. There are many of these. Full Table Scan, Index Unique Scan, Index Range Scan, Index Skip Scan, Index Fast Full Scan, Index Join Scan, Bitmap Conversion, Star Schema Transformation, Bitmap Index Range Scans, Bitmap Index Merge, etc... but basically there’s 5 common ones which we use all of the time so for simplicity, so $5 \times 5 \times 5 \times 5$ is 3,000 possibilities. $(125 \times 81) = 10,125 \times 3,000 = \text{over } 30,000,000$ possible execution plans.

And the optimizer has 2,000 guesses and stops. So the optimizer always gets complex SQL wrong. Sometimes it changes its mind, and sometimes that’s bad. Very very bad.

--> Understand it very clearly, Explain Plan gives an idea , what might happen with query.

=====

The purpose of Performance tuning is as follows.

Performance tuning is the process of optimizing a system or application to ensure it runs efficiently and effectively. The main purposes include:

- **Improved Speed:** Enhancing response times and processing speeds to provide a better user experience.
- **Resource Optimization:** Making better use of hardware resources, such as CPU, memory, and storage, to reduce costs and improve performance.
- **Scalability:** Ensuring that the system can handle increased loads or user traffic without degradation in performance.
- **Reliability:** Reducing the likelihood of errors and crashes, ensuring that the system operates smoothly under various conditions.
- **Maintainability:** Streamlining code and processes to make future updates and maintenance easier.
- **User Satisfaction:** Delivering a faster and more responsive application to enhance user engagement and satisfaction.

Before starting to solve the performance issue, you need to determine what the problems are and how to solve them.

The solution of performance problems are done in the 2 categories.

Instance Tuning

- Tuning memory components
- Tuning space usage and I/O
- DBA help here

SQL Tuning

- Identifying and tuning SQL statements by influencing the optimizer
- Managing change
- SQL Performance Management

For SQL tuning. Find the SQL and check the explain plan. We found the sql from the AWR report.

AWR REPORT: The awrrpti.sql SQL script generates a report (in HTML or text) that displays statistics and give idea of what might happen with the query.

An Automatic Workload Repository (AWR) report is generated by comparing two AWR snapshots.

- Login to the database server.
- Set the ORACLE_HOME environment variable.
- Change the directory to \$ORACLE_HOME/rdbms/admin.
- Login to SQL-PLUS with some privileged user like SYS.
- Execute the script 'awrrpt.sql' like @awrrpt.sql;. This script resides at \$ORACLE_HOME/rdbms/admin location.
- On executing the script, first it will ask to input the value for 'REPORT_TYPE'. 'HTML' is default value which is good so just press Enter key to accept the default value.
- Next input will be asked for 'NUM_DAYS'. Based on input value, it will list the stored snaps for those many days. For example if 1 is entered then all AWR snaps stored in database in last 1 day will be listed along with their snap start times.
- After listing the corresponding AWR snaps, it will ask to input Begin Snap-Id. For example if you want to generate a AWR report from 2 PM to 3 PM then you should enter SNAP-ID, which is generated at 2 PM. Also note that many times the snap will not be available for time that you are looking for then in that case try to use the closest snap-id available.
- Now it will ask for End Snap then enter the Snap-id for 3 PM. Also note that many times the snap will not be available for time that you are looking for then in that case try to use the closest snap-id available.

DB Name

between what time report

WORKLOAD REPOSITORY report for

DB Name	DB Id	Instance	Inst num	Startup Time	Release	RAC
DEVECI	2517056649	DEVECI	1	22-Feb-19 03:02	11.2.0.4.0	NO

Host Name	Platform	CPUs	Cores	Sockets	Memory (GB)
DEVECI001	Microsoft Windows x86 64-bit	8	8	8	16.00

Snap Id	Snap Time	Sessions	Cursors/Session
Begin Snap:	21129	24-Apr-19 04:00:45	55
End Snap:	21329	02-May-19 12:00:46	74
Elapsed:	12,000.02 (mins)		
DB Time:	2,227.04 (mins)		

Report Summary

Load Profile

	Per Second	Per Transaction	Per Exec	Per Call
DB Time(s):	0.2	0.1	0.00	0.00
DB CPU(s):	0.1	0.0	0.00	0.00
Redo size (bytes):	51,425.1	15,210.0		
Logical read (blocks):	11,088.9	3,279.8		
Block changes:	320.7	94.9		
Physical read (blocks):	2,747.8	812.7		
Physical write (blocks):	23.6	7.0		
Read IO requests:	78.6	23.2		
Write IO requests:	5.8	1.7		
Read IO (MB):	21.5	6.4		
Write IO (MB):	0.2	0.1		
User calls:	660.0	195.2		

Go to the main report section and from the main report go to SQL Statistics.

Shared Pool Statistics

	Begin	End
Memory Usage %:	63.06	59.62
% SQL with executions>1:	98.17	82.74
% Memory for SQL w/exec>1:	97.25	82.52

Main Report

- [Report Summary](#)
- [Wait Events Statistics](#)
- [SQL Statistics](#)
- [Instance Activity Statistics](#)
- [IO Stats](#)
- [Buffer Pool Statistics](#)
- [Advisory Statistics](#)
- [Wait Statistics](#)
- [Undo Statistics](#)
- [Latch Statistics](#)
- [Segment Statistics](#)
- [Dictionary Cache Statistics](#)
- [Library Cache Statistics](#)
- [Memory Statistics](#)
- [Streams Statistics](#)
- [Resource Limit Statistics](#)
- [Shared Server Statistics](#)
- [init.ora Parameters](#)

focus here/
click on this

When we click on SQL: Statistics, it will show all data related to SQL. we choose the option, sql ordered by elapsed time. It will show all sql, which are executed and took time.

SQL ordered by Elapsed Time

SQL Statistics

- SQL ordered by Elapsed Time
- SQL ordered by CPU Time
- SQL ordered by User I/O Wait Time
- SQL ordered by Gets
- SQL ordered by Reads
- SQL ordered by Physical Reads (UnOptimized)
- SQL ordered by Executions
- SQL ordered by Parse Calls
- SQL ordered by Sharable Memory
- SQL ordered by Version Count
- Complete List of SQL Text

[Back to Top](#)

SQL ordered by Elapsed Time

- Resources reported for PL/SQL code includes the resources used by all SQL statements called by the code.
- % Total DB Time is the Elapsed Time of the SQL statement divided into the Total Database Time multiplied by 100
- %Total - Elapsed Time as a percentage of Total DB time
- %CPU - CPU Time as a percentage of Elapsed Time
- %IO - User I/O Time as a percentage of Elapsed Time
- Captured SQL account for 57.9% of Total DB Time (s): 133,622
- Captured PL/SQL account for 5.5% of Total DB Time (s): 133,622

Elapsed Time (s)	Executions	Elapsed Time per Exec (s)	%Total	%CPU	%IO	SQL Id	SQL Module	SQL Text
10,311.03	36	286.42	7.72	36.01	66.38	a2xym4cfx0356	deveci.exe	SELECT KST_ID, ART_ID, TO_CHAR...
4,283.15	594,829	0.01	3.21	100.18	0.10	2j75r3hvm813p	deveci.exe	select sum(l.libp_menge*v.vpk...
3,515.31	3	1,171.77	2.63	84.73	15.08	9v8t560dyb2zx	w3wp.exe	WITH sales AS (SELECT daykst...
2,702.43	2	1,351.21	2.02	46.17	55.10	5zruc4v6y32f9	DBMS_SCHEDULER	DECLARE job BINARY_INTEGER := ...
2,674.78	17	157.34	2.00	84.42	14.90	2frDEVECic3y66dyc	w3wp.exe	WITH sales AS (SELECT daykst...
2,189.86	8	273.73	1.64	68.01	3.28	88bftbt7a5p7z	exp.exe	SELECT /*+NESTED_TABLE_GET_REF...
2,061.77	8	257.72	1.54	71.67	28.15	14d95kacx7mfd	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,037.22	8	254.65	1.52	72.08	27.08	aaq82k7ch9kzh	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,021.51	8	252.69	1.51	72.89	26.28	f3st1tkq7xqsj	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,018.51	8	252.31	1.51	72.84	26.61	9dz97myx1zcj4	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,010.26	8	251.28	1.50	72.66	26.43	2xsntepw35k49	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,009.64	8	251.21	1.50	72.58	26.53	848x8m2388qtf	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
2,005.52	8	250.69	1.50	73.19	26.02	f5u8n0dvrn5wb	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
1,980.23	8	247.53	1.48	74.06	25.02	7cabrvujhb74m	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
1,786.74	7	255.25	1.34	71.76	27.38	f4a5nrjfqf2yy	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...
1,778.71	7	254.10	1.33	72.60	26.96	228tsdaudryx	deveci.exe	SELECT ptt.DAY_DATE, ptt.KST_I...

=====

=====

What is Cost ?

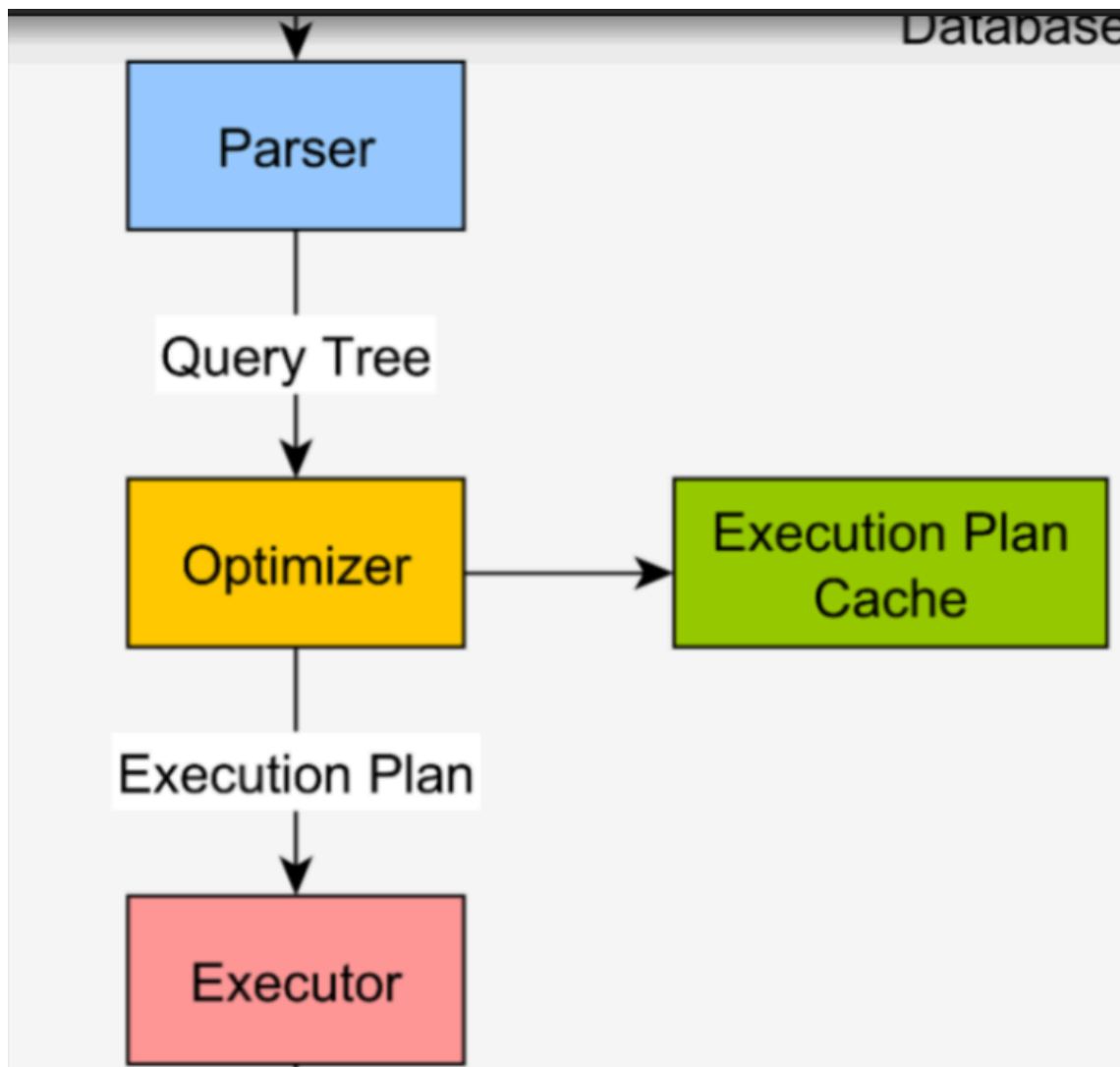
The Oracle Optimizer is a cost-based optimizer. Cost of the operation as estimated by the optimizer's cost-based approach. For statements that use the rule-based approach, this column is null. The Optimizer selects the execution plan with the lowest cost, where cost represents the estimated resource usage for that plan. The lower the cost the more efficient the plan is expected to be. The optimizer's cost model accounts for the IO, CPU, and network resources that will be used by the query.

CARDINALITY

Estimated by the cost-based approach of the number of rows accessed by the operation. It is the ratio of total distinct values with the number of records in the table.

When we run any SQL statement, it goes through 3 phases :

- the Parser,
- the Optimizer,
- the Executor.



Parser

The Parser checks the SQL statement and ensures its validity. The statements are verified both syntactically (the statement keywords must be properly spelled and following the SQL language guidelines) and semantically (the referenced tables and column do exist in the database).

Optimizer

For a given syntax tree, the database must decide the most efficient data fetching algorithm. When finding an optimal execution plan, the Optimizer might evaluate multiple options, and, based on their overall cost, it chooses the one requiring the least amount of time to execute. Which order, which joining method and which access path. Plan Hash value generated and stored.

Executor

From the Optimizer, the execution plan goes to the Executor where it is used to fetch the associated data and build the result set.

The Executor simply runs the execution plan which is like a runtime-generated program that tells the Executor how to fetch the data the client requires.

When an SQL statement is executed, the database engine follows a set of steps to process the statement and produce the desired result. Here is an example of how a SELECT statement is executed:

Consider the following SQL statement:

SELECT * FROM customers WHERE age > 30 ORDER BY last_name;

1. **Parsing:** The database engine first parses the statement to make sure it is syntactically correct and assigns an execution plan.

2. **Authorization Check:** The database engine checks the user's permissions to ensure that they have the necessary privileges to execute the statement.

3. **Optimization:** The database engine then optimizes the execution plan by examining the statement's query paths, indexing schemes, join methods, and other aspects to create an efficient plan.

4. **Execution:** With the optimized plan in place, the database engine executes the query and returns the result set.

5. **Post-processing:** If necessary, other post-processing steps might be performed, such as data type conversion or data formatting.

-these steps ensure that the SQL statement is executed accurately and efficiently, producing the desired result set.

=====

Parsing is primary phase in the SQL execution.

PARSING BASICS:

Whenever a SQL statement is executed, Oracle Engine performs below actions on it:

- 1) **Syntactic** : Checks the syntax of the query
- 2) **Semantic** : Checks that all objects exist and are accessible
- 3) **Optimization** : Determines the optimal access path for the query to take. With the Rule Based Optimizer (RBO) it uses a set of heuristics to determine access path. With the Cost Based Optimizer (CBO) we use statistics to analyze the relative costs of accessing objects.
- 4) **QEP Generation** : QEP = Query Evaluation Plan
- 5) **QEP Execution** : QEP = Query Evaluation Plan

Verifying if the statement is already available in the shared pool. Oracle engine will calculate the hash value for the SQL statement and look in If statement is not already present then allocate shared memory and create a cursor in the shared pool.

TYPES OF PARSES

HARD PARSE: It means that statement is not available in shared memory or this is a brand new statement that user is trying to execute. If your shared pool is small then also hard parse may be required as the old statement got aged out the shared pool. All the steps mentioned above for parsing need to be done for this situation. Hard parsing requires extra system resources. This is also known as 'Library Cache Miss'.

SOFT PARSE: It means that statement was executed earlier and was already parsed and is available in memory. So Oracle need to do steps 1-3 only as mentioned above since rest of the tasks were already done earlier. It is like work hard once and reap benefits multiple times. This is also known as 'Library cache Hit' as you got the statement parsed and available to use in the Library cache.

The parse operation is to see if the statement we are currently parsing has already in fact been processed by some other session. If it has, we may be in luck here, we can skip the next two steps in the process, that of optimization and row source generation.(That is time consuming, as we do which order, which method, which access path). If we can skip these next two steps in the process, we have done what is known as a Soft Parse.

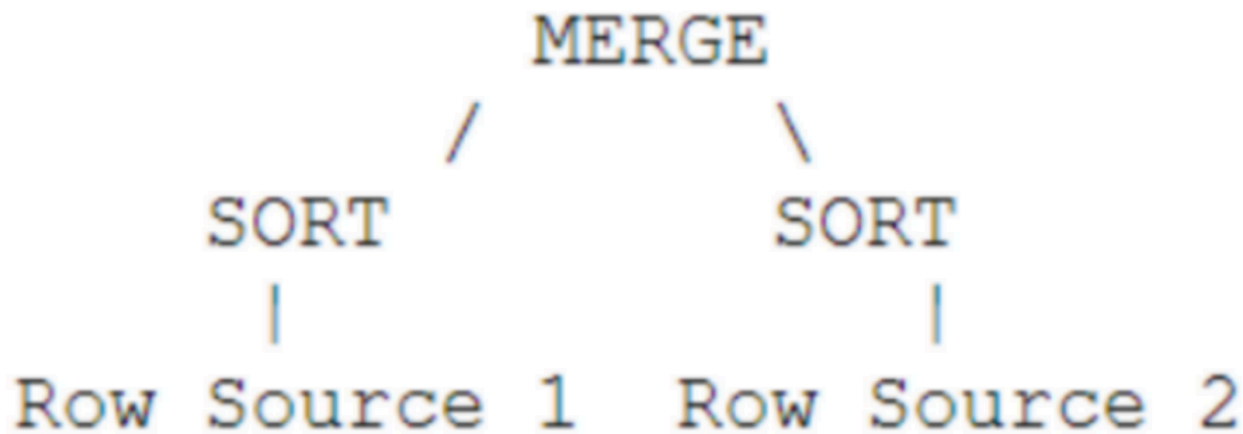
A shorter process to get our query going. If we cannot, if we must do all of the steps, we are performing what is known as a Hard Parse. we must parse, optimize, generate the plan for the query. This distinction is very important. When developing our applications we want a very high percentage of our queries to be Soft Parsed, to be able to skip the optimize/generate phases as they are very CPU intensive. If we have to Hard Parse a large percentage of our queries, our system will function slowly and in some cases.

The way this sharing of SQL in Oracle is accomplished is via the shared pool, a piece of memory in the SGA maintained by Oracle. Oracle gives us the shared_pool_size parameter to cache SQL so that we don't have to parse, over-and-over again.

=====

Join Types

- Sort Merge Join (SMJ)
- Nested Loops (NL)
- Hash Join



Rows are produced by Row Source 1 and are then sorted. Rows from Row Source 2 are then produced and sorted by the same sort key as Row Source 1. Sorting is an expensive operation, especially with large tables. Because of this, SMJ is often not a particularly efficient join method.

Sort Merge joins are useful when the join condition between two tables is an in-equality condition such as $<$, $<=$, $>$, or $>=$.

[Nested Loops \(NL\)](#)

Nested loops joins are useful when small subsets of data are being joined and if there is an efficient way of accessing the second table. For every row in the first table (the outer table), Oracle accesses all the rows in the second table (the inner table) looking for a match.

```
// outer loop
for (every row in the outer table)
{
    // inner loop
    for (every row in the inner table)
    {
        Check if you have a match
    } ..
}
```

[Hash Join](#)

This join is more efficient in theory than NL & SMJ. Only accessible via the CBO.

In a hash join, the Oracle database does a full-scan of the driving table, builds a RAM hash table, and then probes for matching rows in the other table. For certain types of SQL, the hash join will execute faster than a nested loop join, but the hash join uses more RAM resources.

Hash joins are used for joining large data sets. The Optimizer uses the smaller of the two tables or data sources to build a hash table, based on the join key, in memory. It then scans the larger table and performs the same hashing algorithm on the join column(s).

--> A hash join is an operation that performs a full-table scans on the smaller of the two tables (the driving table) and then builds a hash table in RAM memory. The hash table is then used to retrieve the rows in the larger table.

Some queries will perform faster with NESTED LOOPS joins, some with HASH joins, while others favor sort-merge joins. It is difficult to predict what join technique will be fastest

Example of DECK OF CARDS

Suppose, I have 2 deck of cards. Like 2 tables. For every card in this first or outer deck, I am gonna look for a card with the same value and suit in this second or inner deck. So what we'll do is we'll take the top card from here. That's the six of clubs. I'll then search through all the cards in this second deck until - eventually - we get to there we go there's six of clubs. So we have a join there. We've joined those cards. We can then go back to our first deck.

So for every card in this first or outer deck I'm going to look through every single card in this second or inner deck. **This is the basis of a nested loop join.**

For every row in the first table, we will query the second table looking for matching rows. Clearly in this case it is very slow. We're going to read every row in the second table once for every row in the first. That's a lot of work. So how can we make combining the decks faster? Well one trick is to sort the decks first. So now we have two ordered decks and I'm going to join them again. So we'll start with the first card in the outer deck. Suppose, That's the two of clubs. We then go to the second inner deck and that also has the two clubs as the first card. So we've got a join. I can then go to the next card in this second deck, that's the three of clubs. This no longer matches. So because this second deck is sorted, we now know that there can be no other cards with the value two of clubs in the second deck. So we can stop processing now. **This is the basis of a merge join.**

First we sort the tables on the join columns and then walk down them bit by bit comparing the rows. The key thing is because the rows are sorted we can stop processing once we've looked at a value in the second table which is greater than the value of the current row in the first table. So combining the decks with a merge join is faster than nested loops. But you need to sort them first. And sorting is slow. So how can we do any better?

what I'm going to do is first look at all the cards in my outer deck. For each one what I'll do is look at its value and apply a hash function and build a hash table on it. For every card, will give a hash value. Once I've read them all I will then go to the second deck and apply the same hash function to the value. So suppose we've got the Queen of Hearts, apply the same hash function and then lookup - probe - the hash table we built for matching values. in other table. We've got a matching entry here so we have a join. **this is the basis of a hash join.**

This first reads all the rows in one table, applying a hash function to the join criteria to build the hash table. It then walks through all the rows in the second table, applying the same hash function and looking up - probing - the hash table to see if there is a matching value in the hash table we've just built. So we've got our three different approaches: nested loops, merge-join, hash join.

Some queries will perform faster with NESTED LOOPS joins, some with HASH joins, while others favor sort-merge joins. It is difficult to predict what join technique will be fastest

=====

Difference between Nested Loop Join and Hash Join :

S.No.	Nested Loop Join	Hash Join
1.	It is processed by forming an outer loop within an inner loop after which the inner loop is individually processed for the fewer entries that it has.	It is specifically used in case of joining of larger tables.
2.	The nested join has the least performance in case of large tables.	It has best performance in case of large and sorted and non-indexed inputs.
3.	There are two phases in this, outer and inner loop processing.	The two phases in this are build and probe.
4.	Steps involved include identifying an outer driving table and assigning the inner table to it, and processing the rows of inner table for every outer table row.	The steps involved are building a Hash table on a small table. It is used to probe the hash value of the Hash table is applicable for each element in the second row.
5.	Index range scan is done here.	Full-table scan of the smaller table is done in case of hash join.
6.	This uses lesser RAM resources.	It uses more RAM resources.
7.	It is the most common type of join.	It is not as common as the nested loop join.
8.	Least number of comparisons are required in case of nested loop join.	It needs more comparisons than the nested loop join thereby using more RAM resources.
9.	It is the fastest join algorithm due to least number of comparisons.	It is not as fast due to more number of comparisons.
10.	It is better than all other types of join for small transactions and small data.	It is not as good as nested loop join in case of smaller data.
11.	It is of three types, namely, nested loop join, indexed nested loop join and Temporary indexed nested loop join.	Its types are classic hash join, Grace hash join, hybrid hash join, hash anti join, hash semi-join, recursive hash join and hash bailout.

S.No.	Nested Loop Join	Hash Join
12.	It is not automatically selected.	This join is automatically selected in case there is no specific reason to adopt other types of join algorithms. It is also known as the go-to guy of all the join operators.

Difference between Nested Loop Join and Sort Merge Join :

S.No.	Nested Loop Join	Sort Merge Join
1.	It is processed by forming an outer loop within an inner loop after which the inner loop is individually processed for the fewer entries that it has.	It is usually used to join two independent sources of data represented in a table.
2.	The nested join has the least performance in case of large tables.	It is better than nested join in case of performance in large tables.
3.	There are two phases in this, outer and inner loop processing.	It consists of 2 phases consisting the sort operation and merge operation.
4.	Steps involved include identifying an outer driving table and assigning the inner table to it, and processing the rows of inner table for every outer table row.	The first row from the first table and second row from the table is taken, if it is not the end then, the selected rows are checked for the merger. If they can be merged, the merged row is returned else next rows are taken from the tables and the steps are repeated until the rows are exhausted.
5.	It is not as fast as sort merge join in case of sorted tables.	It is the fastest join operation in case of sorted tables. This is because it uses merge phase and sort phase, where, if sort is already previously done, then merge is the fastest operation.
6.	It is of three types, namely, nested loop join, indexed nested loop join and	It does not have further classifications.

S.No.	Nested Loop Join	Sort Merge Join
	Temporary indexed nested loop join.	
7.	It is the most common type of join.	It is not as common as the nested loop join.
8.	Least number of comparisons are required in case of nested loop join.	It needs more comparisons than the nested loop join (generally).
9.	It is the fastest join algorithm due to least number of comparisons.	It is not as fast due to more number of comparisons.
10.	It is better than all other types of join for small transactions and small data.	It is not as good as nested loop join in case of smaller data.

```
=====
=====
```

HINTS

Hints are used to change the execution plan of the sql. supported by CBO. We can write like `/*+ hint */`

It's a way to override the default query optimization in the DBMS. Influence the execution plan of query

Why Use Hints?

Oracle optimizer may not always choose the best execution plan

Using hints may improve the performance by changing the execution plan oracle takes.

Types of Hints

Approach Hints

ALL_ROWS Minimizes total resource consumption.

Results will be returned only after all processing has been completed

FIRST_ROWS(n) Minimized response time, or minimal resource usage to return the first n rows.

Access Hints

FULL(table) Chooses a full table scan for the table, even if there is an index available.

INDEX(table index index...) Chooses an Index scan for the table.

Join hints

ORDERED tables are joined in the order in which they appear in the FROM clause.

LEADING(table) specified table is the first table used in the join order.

USE_HASH(table table) Tables are joined using a hash join. Smaller table is used to make a hash table on join key. The larger table is

scanned using hash table to find joined rows.

USE_NL(table table) Joins tables using nested loops join, using specified table as inner join. For every row in outer table, oracle accesses

every row in inner table.

USE_MERGE(table table) Joins tables using a sort-merge join. Sorted list are made and then merged together. Best if tables are already

sorted.

Misc. Hints

APPEND Data is appended to the end of table, rather than using existing space.

Parallel enables a SQL statement to be processed by multiple threads or processes simultaneously

The parallel hint is one of the good optimizer hints which allows you to improve the performance of SQL select statement.

SELECT /*+ parallel(employees,8) */ employee_id, Employee_name FROM hr.employees;

The above statement will run the query with 8 degree of parallelism. Here 8 different processes will open to run the select query fast. Each one of the processes will take the subset of the data.

If the above employees table has 80000 records this means that each process will process 10000 records parallelly rather than serial execution of 80000 records together.

Drawback of Parallel Hint

The PARALLEL hint in Oracle is used to instruct the query optimizer to parallelize the execution of a SQL statement by using multiple parallel execution servers. While it can improve the performance of certain queries, it also comes with some drawbacks and considerations:

- Parallel execution consumes additional system resources, such as CPU, memory, and I/O bandwidth.
- There is an overhead associated with parallelizing a query, including the cost of starting and coordinating parallel processes.
- Parallel execution is not suitable for all types of queries. It is most effective for large data sets and complex queries.
- Parallel execution can introduce concurrency issues such as blocking and deadlocks, especially when multiple sessions are competing for parallel resources.